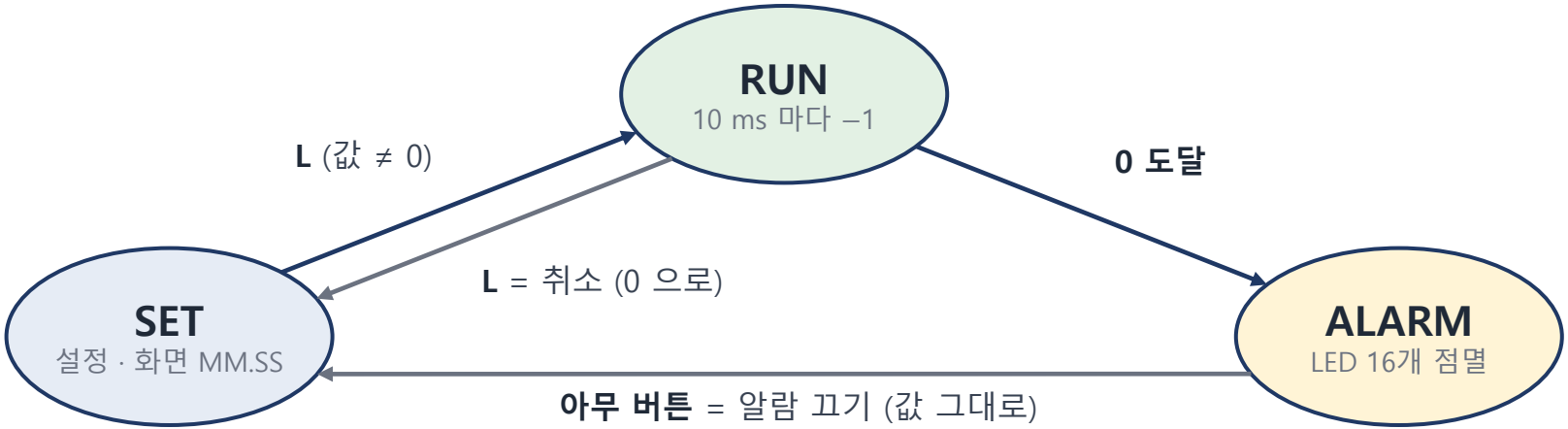
타이머 1. 동작 사양

SET · RUN · ALARM 3상태 FSM — 버튼 4개로 더하고, 시작하고, 끈다



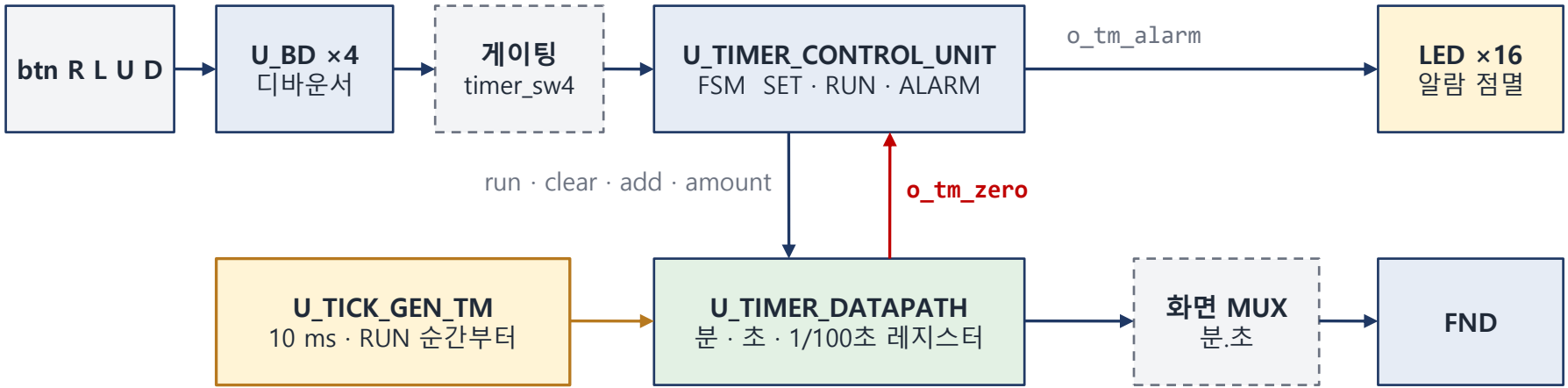
SET 에서 버튼	U	R	D	L
하는 일	+10초	+1분	+10분	시작

상한 59:59 — 더해서 60분 이상이면 59:59 로 멈춤 (화면이 분.초 4자리)

RUN 중 U · R · D 는 무시 · timer_sw4 = 1 이면 다른 화면보다 우선 · 다른 화면에서도 타이머는 계속 감

타이머 2. 구조

컨트롤 유닛(FSM, 언제) + 데이터패스(값) — 0 도달은 데이터패스가 알려 줌



같은 사슬 — 시계 · 스톱워치도 버튼 → 디바운서 → 게이팅 → FSM → 데이터패스 → 화면. 4 kHz 디바운스 박자와 FND 는 셋이 공유

타이머 3. 검증 방법

TB 3개 · 검사 73개를 $display 로 자동 판정 — 사람이 파형을 읽기 전에 PASS/FAIL 이 먼저 나옴

TB	확인하는 것	검사	결과
TM1	더하기 3종 · 초→분 올림 · 59:59 상한 · 0 에서 시작 안 함	21	**PASS**
TM2	실제 10초 카운트다운 · 알람 LED 점멸 · 끄기 · 분 빌림	41	**PASS**
TM3	RUN 중 버튼 무시 · 취소 · 시계 화면에서도 알람 · 화면 우선순위	11	**PASS**

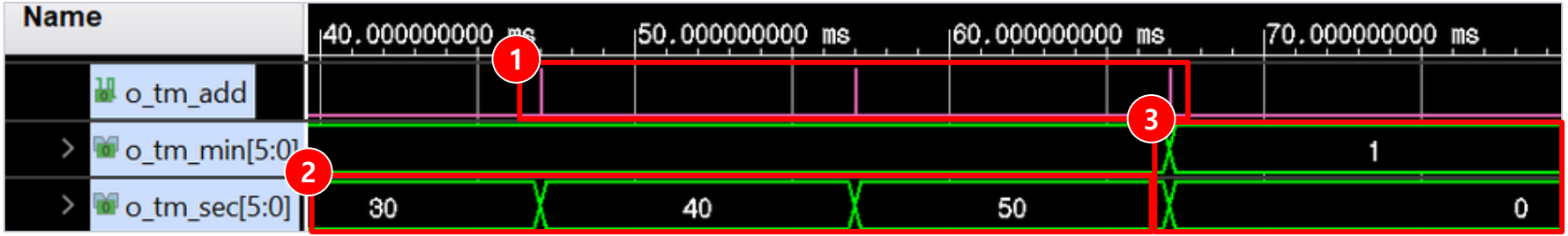
```
# Vivado xsim 콘솔 — 팀 TB (SystemVerilog) 마지막 줄 (뒷부분 항목 생략)
[  455.019 ms] [SUMMARY] TM1 checks=21 fail=0 => PASS
[11597.002 ms] [SUMMARY] TM2 checks=41 fail=0 => PASS
[  547.009 ms] [SUMMARY] TM3 checks=11 fail=0 => PASS
# 같은 시나리오를 수업 Verilog 문법(err 카운트)으로 옮긴 TB
TM1 checks = 21, TOTAL MISMATCH = 0   (TM2 41 · TM3 11 도 0)
```

판정 예 **`if (값 !== 기대값) err = err + 1;  $display("FAIL …");`** → 끝에 TOTAL MISMATCH 출력

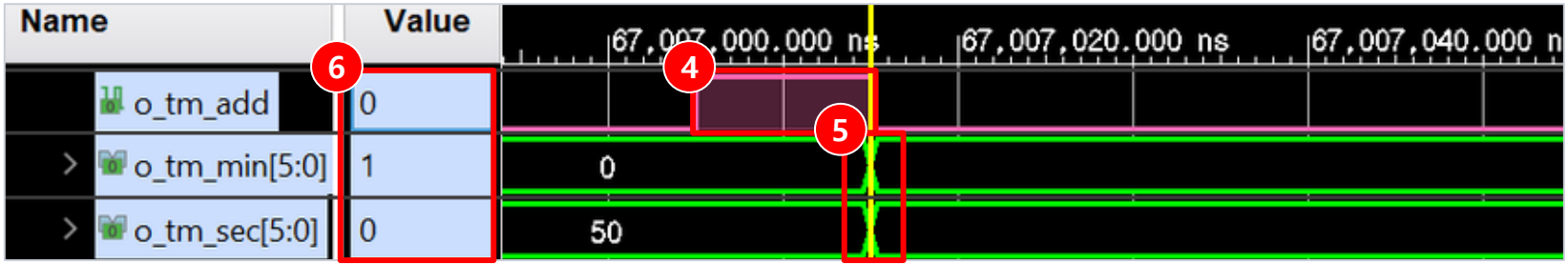
캡처는 10/17 소스, 결과 숫자는 10/18 최종 소스(④ 반영) 기준 — 둘 다 같은 TB 전부 PASS

타이머 검증 1. 더하기 · 상한

결과: U 6번째 — 00:50 + 10초 = 01:00 (60초 아님)



① U 4 · 5 · 6번째 add 펄스 ② 초 30 → 40 → 50 ③ **6번째: 분 1 · 초 0**



④ 6번째 add 펄스 (ns 확대) ⑤ **다음 클럭 00:50 → 01:00** ⑥ 커서 값: 분 1 · 초 0

00:50 + 0:10 = 초 60 → 분 올림 → **01:00**

타이머 검증 1. 더하기 · 상한

결과: 59:50 + 10초 → 59:59 에서 멈춤 (화면이 분.초 4자리라 60:00 불가)



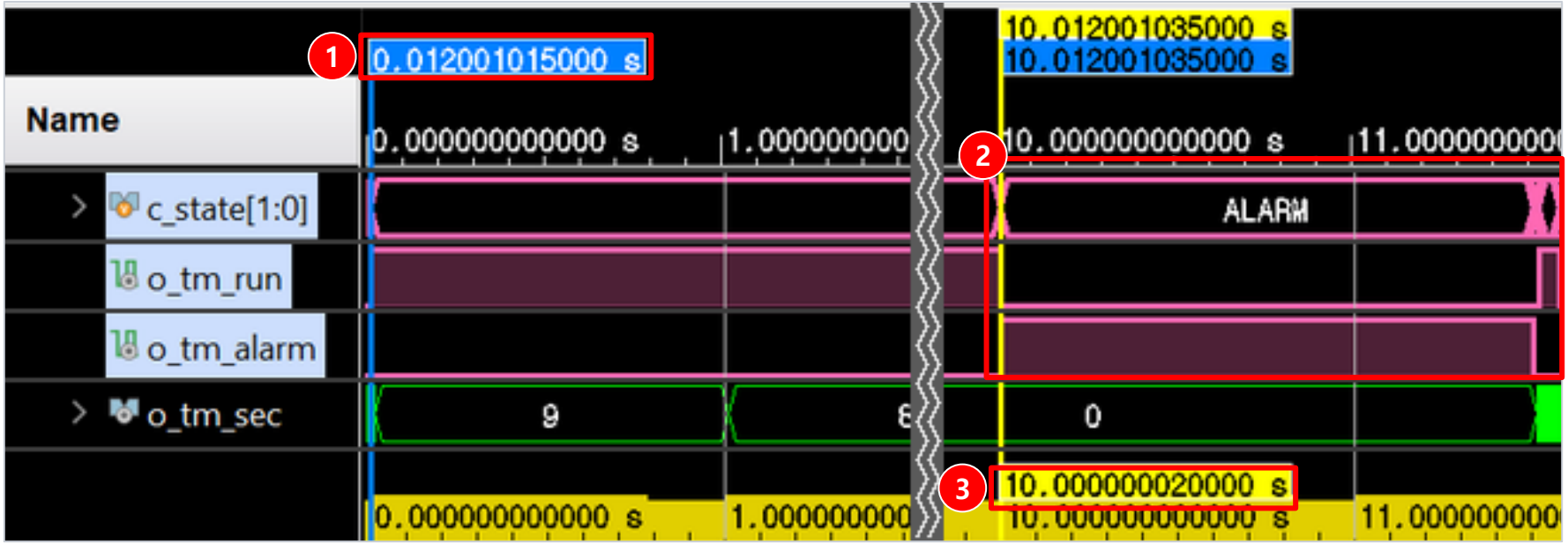
① U: 59:40 → 59:50 ② **U: 59:50 → 59:59** ③ 59:59 유지 (60:00 아님)

59:50 + 0:10 = 60:00 → 상한 적용 → **59:59**

timer_datapath: 더한 값이 60분 이상이면 59:59 로 제한

타이머 검증 2. 카운트다운 · 알람

결과: 00:10 설정 → 시작부터 알람까지 10.000000020 s (= 10 s + 2클럭)

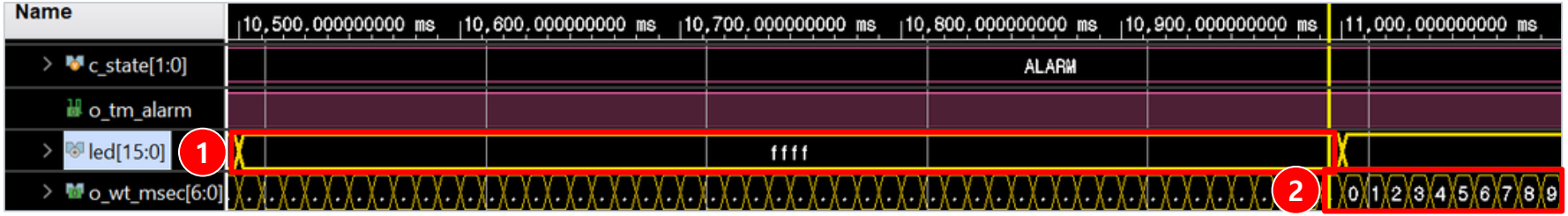


① RUN 시작 커서 0.012 s ② **RUN → ALARM, alarm 1** ③ **간격 10.000000020 s**

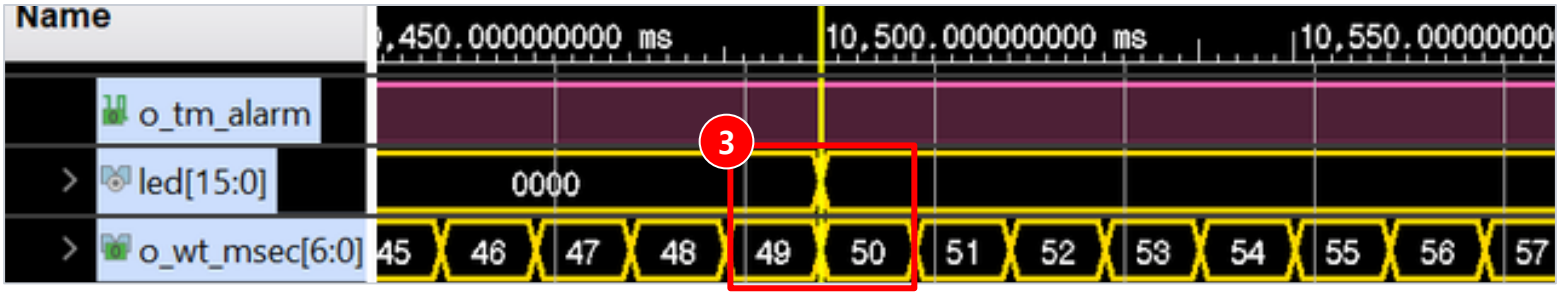
가운데 ≋ = 1 s ~ 10 s 생략 (초 9 → 1 이 1초마다 줄어듦) · After = ④ 소스, 배치 실행 wdb 를 열어 캡처

타이머 검증 2. 카운트다운 · 알람

결과: 알람 중 LED 16개가 0.5 s 켜짐 · 0.5 s 꺼짐 반복



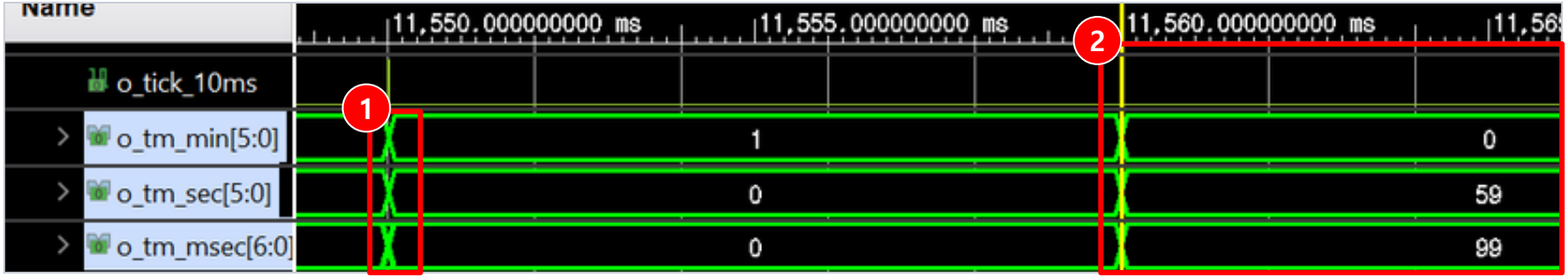
① **led = ffff, 0.5 s 동안 켜짐** ② 시계 1/100초 00 → led 0000 (꺼짐)



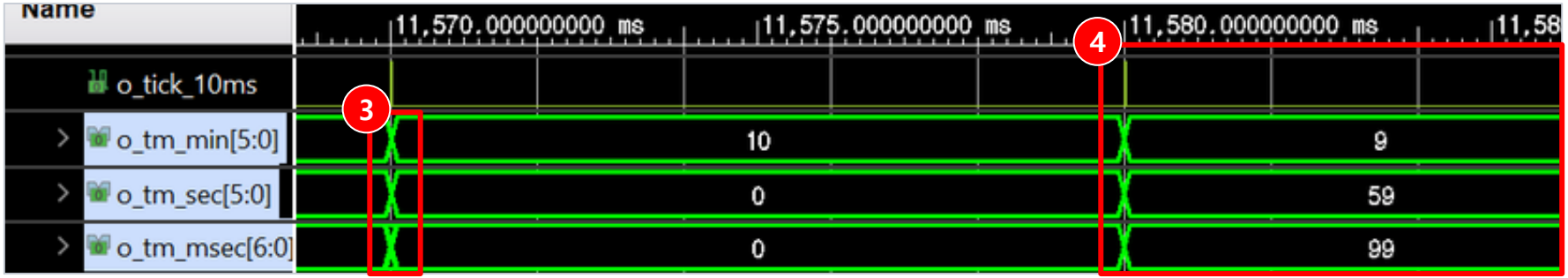
③ **확대: 시계 1/100초 49 → 50 인 클럭에 0000 → ffff**

타이머 검증 2. 카운트다운 · 알람

결과: 빌림 정상 — 01:00.00 → 00:59.99, 10:00.00 → 09:59.99



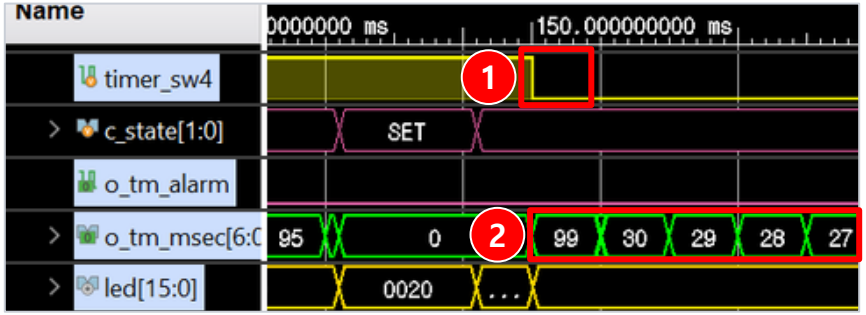
① 01:00.00 넣음 ② **다음 tick → 00:59.99**



③ 10:00.00 넣음 ④ **다음 tick → 09:59.99**

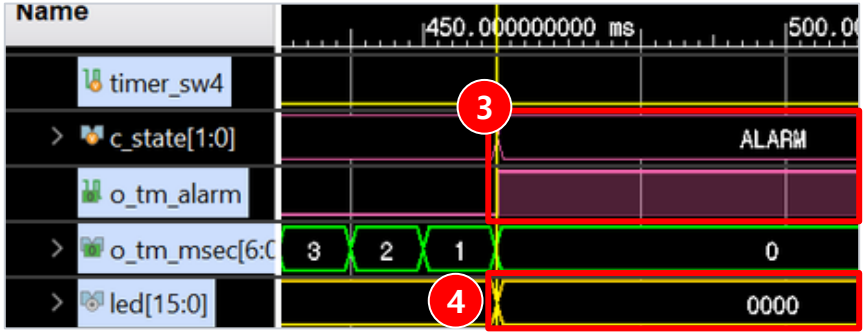
타이머 검증 3. 취소 · 백그라운드 · 우선순위

결과: 시계 화면으로 바꿔도 타이머는 계속 줄고, 0 이 되면 알람



① timer_sw4 1 → 0 (시계 화면)

② 값은 계속 줄어듦 99 → 30 → 29 …



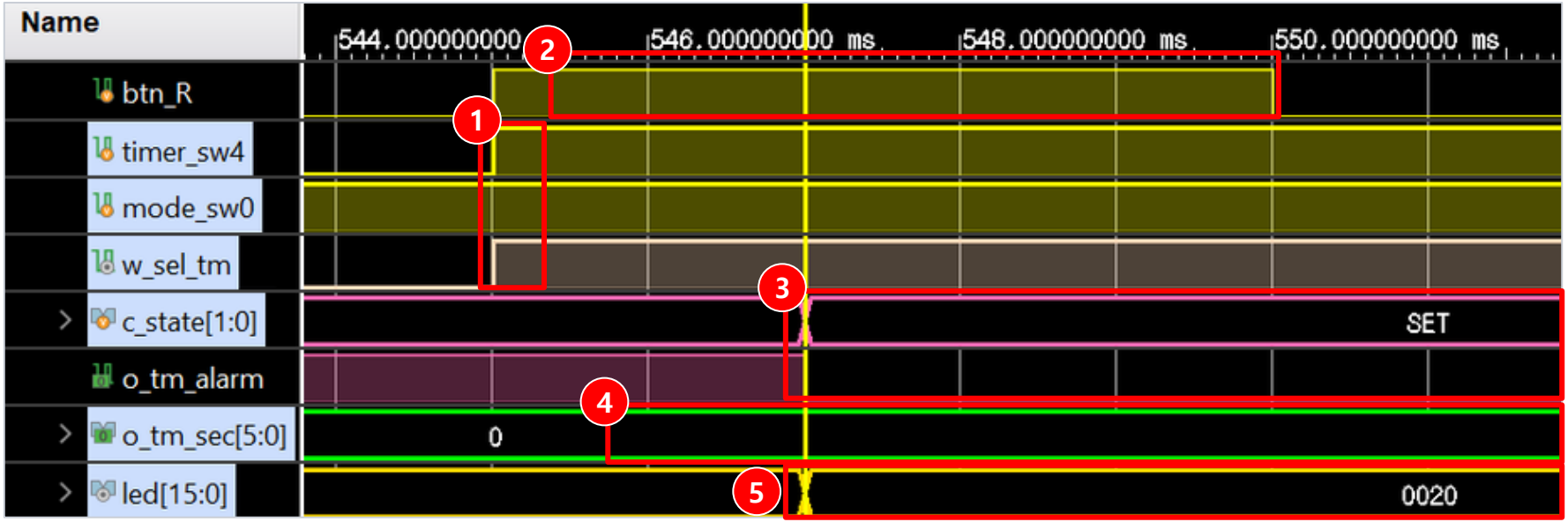
③ **RUN → ALARM, alarm 1**

④ LED 점멸 (이 순간은 꺼진 쪽 0000)

~150 ms sw4 → 0 (시계 화면) — **약 310 ms 동안 화면은 시계, 타이머는 계속 감소** → **~460 ms** 0 → ALARM

타이머 검증 3. 취소 · 백그라운드 · 우선순위

결과: sw4 · sw0 둘 다 1 이면 타이머 화면, 알람 중 R 은 알람만 끄고 값은 그대로

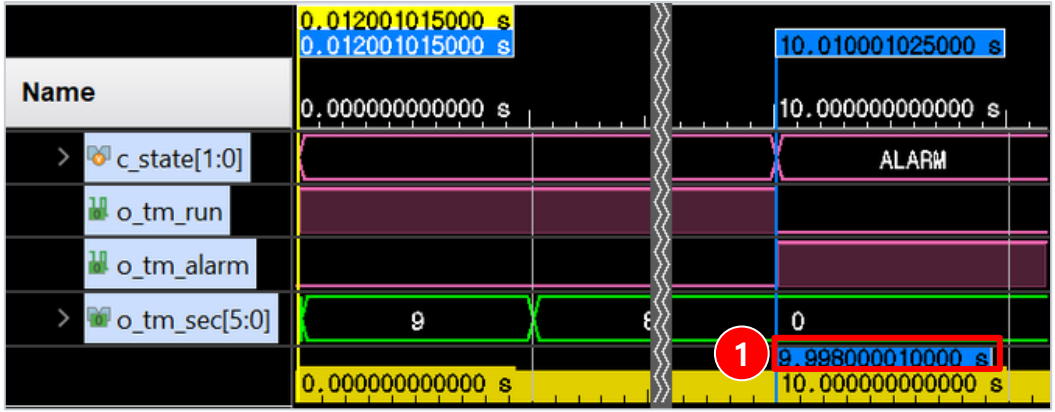


① sw4 1 (sw0 도 1) → 타이머 화면 ② R 누름 ③ **ALARM → SET, alarm 0**

④ **값 그대로 (1분 안 더함)** ⑤ LED 점멸 끝 (0020)

트러블슈팅 — 10초 타이머가 9.998초에 울림

① 현상 · ② 원인: 100 Hz tick 하나를 시계 · 스톱워치 · 타이머가 같이 씀

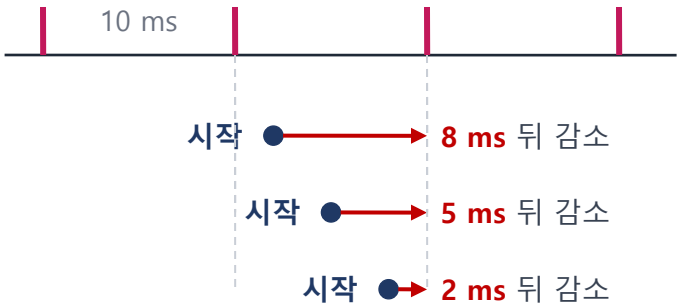


① 현상 RUN 시작 → ALARM 간격 **9.998000010 s**

10/17 캡처 · 가운데 ≈ = 1~9 s 생략

② 원인

공유 tick (리셋부터 항상 10 ms 박자)



시계: 시작이 없어 상관없음

스톱워치 · 타이머: 누른 순간이 0 이어야 함

측정 tb_tm_2 P1 9,998 ms · 시작 시각을 3 ms 씩 바꿔 돌린 TB 에서 첫 감소 8 / 5 / 2 ms

트러블슈팅 — 10초 타이머가 9.998초에 울림

③ 해결: 4가지 구조를 같은 보드 · 같은 TB 로 비교 → ④ 선택

판	구조	LUT	FF	WNS	시작 → 첫 tick
①	수업 방식: 디바운서마다 분주 + 데이터패스마다 tick	403	292	+3.47	매번 10 ms
②	기존 팀 판: 4 kHz 공유 + 100 Hz 1개 셋이 공유	327	186	+3.22	**최대 10 ms 일찍**
③	참고: ② + 4 kHz 를 enable 로	317	172	+2.94	② 와 같음
④	**4 kHz 공유 + 100 Hz 기능마다 따로 (RUN 순간부터)**	**356**	**228**	**+3.63**	**매번 정확히 10 ms**

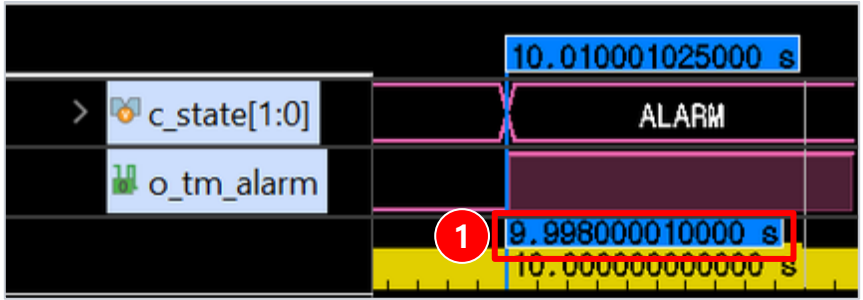
왜 ④ 100 Hz 는 "언제 시작했나" 가 기능이라 나누고, 4 kHz 는 디바운서가 "8번 연속 1" 만 봐서 시작 시점이 없으니 공유 유지

대가 ② 보다 FF +42 (tick 생성기 2개 × 21) · ① 보다 LUT −47 · FF −64 (분주 공유 이득은 그대로)

Vivado 2022.2 · xc7a35t · 구현(place & route) 후 · 같은 xdc. WNS 는 모두 양수(100 MHz 만족)라 우열로 보지 않음. ①③ 은 비교용으로 만든 판(팀 코드 아님), 측정은 배치 실행

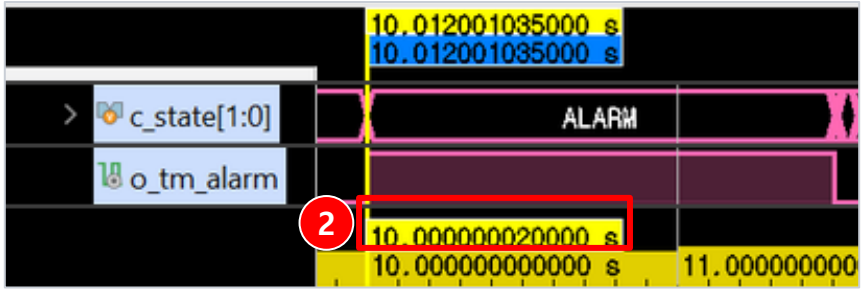
트러블슈팅 — 10초 타이머가 9.998초에 울림

④ 결과: 2 ms 이른 알람 → 10 s + 20 ns (2클럭 고정 지연), 고친 뒤 잔여 2건도 잡음



Before 간격 **9.998000010 s**

시작 시점에 따라 **최대 10 ms 일찍** (이 캡처는 2 ms)



After 간격 **10.000000020 s**

= 10 s + **20 ns** = 2클럭 고정 (매번 같음)

```
# 잔여 2건 — 같은 TB 를 고치기 전 / 고친 뒤 소스로 (콘솔 요약)
[before] 알람 뒤 재시작 → 첫 tick = 9999980 ns   FAIL
[before] tick 클럭에 STOP → msec = 4 (10 ms 잃음)  FAIL
[after ] 알람 뒤 재시작 → 첫 tick = 10000000 ns
[after ] tick 클럭에 STOP → msec = 5   TOTAL MISMATCH = 0
```

고친 것: 타이머 tick clear = 취소 | 알람 · 스톱워치 데이터패스의 중복 AND 삭제 (tick 생성기가 이미 run 을 봄)

검증 신뢰도 — TB 가 틀린 설계를 정말 잡는가

설계 복사본에 bug 를 일부러 넣고 같은 TB 로 돌림 (변이 테스트)

일부러 넣은 bug	TB	결과
timer_datapath: 59:59 상한 조건을 끔	TM1	**FAIL 7개**
timer_control_unit: RUN 에서 U 를 받아 +10초	TM3	**FAIL 1개**
원래 설계	TM1 · TM3	**MISMATCH 0**

```
# bug 넣은 설계로 돌린 콘솔
[310015010 ns] FAIL P5 TB1 59:50 + U exp 59:59.00 got 60:0.0
TM1 checks = 21, TOTAL MISMATCH = 7
[50003010 ns] FAIL P1 TC2 U/R/D during RUN exp (5989,5999) got 6996
TM3 checks = 11, TOTAL MISMATCH = 1
```

→ MISMATCH 0 은 "검사가 엉성해서 0" 이 아니라 설계가 사양대로라는 뜻